

# TypeScript - II



Session 2



- Lớp và đối tượng
- Lập trình hướng đối tượng với TypeScript
- Interface
- Generics
- Namespace & Module
- Ambient

- **Class:** Là một khuôn mẫu lý thuyết để mô tả về các đối tượng thuộc cùng một loại. Lớp là bản thiết kế để từ đó ta khởi tạo các đối tượng.
- **Bên trong class sẽ bao gồm các thành phần sau:**
  - **Fields:** Là các biến được khai báo bên trong lớp. Đây chính là các thuộc tính của lớp.
  - **Methods:** Là các hành động mà lớp cung cấp.
  - **Constructors:** Là method đặc biệt được dùng để khởi tạo các đối tượng của lớp.
- TypeScript là ngôn ngữ kịch bản hướng đối tượng, do đó nó hỗ trợ đầy đủ các đặc điểm của OOP bao gồm classes, interfaces, polymorphism, data-binding v.v... JavaScript **ES5** và các phiên bản trước đó không hỗ trợ classes. TypeScript hỗ trợ đầy đủ OOP từ phiên bản **ES6** trở đi. TypeScript hỗ trợ các classes vì đây là ngôn ngữ được dựa trên phiên bản ES6 của JavaScript.

- Cú pháp định nghĩa class

- Để định nghĩa lớp, ta dùng từ khóa class trong TypeScript. Cú pháp như sau:

```
class <class_name>{  
    field;  
    method;  
}
```

- Ví dụ:

```
class Student {  
    studCode: number;  
    studName: string;  
  
    constructor(code: number, name: string) {  
        this.studName = name;  
        this.studCode = code;  
    }  
  
    getGrade() : string {  
        return "A+";  
    }  
}
```

- Khởi tạo đối tượng

- Để tạo đối tượng của lớp, ta sử dụng toán tử **new** sau đó là **class name**. Toán tử new sẽ cấp phát bộ nhớ để lưu trữ đối tượng lúc runtime. Các đối tượng sẽ được lưu trong vùng bộ nhớ heap.
- Cú pháp khởi tạo đối tượng như sau.
  - ◆ Từ khóa new: Dùng để khởi tạo đối tượng trong bộ nhớ.

```
let object_name = new class_name(parameter)
```

◆ Ví dụ:

```
//Creating an object or instance  
let obj = new Student();
```



- Khởi tạo đối tượng
  - Có 3 cách khởi tạo đối tượng như sau:
  - Bằng biến tham chiếu (reference variable)

```
//Creating an object or instance  
let obj = new Student();
```

```
//Initializing an object by reference variable  
obj.id = 101;  
obj.name = "Virat Kohli";
```

- Bằng method.

```
//Defining a Student class.
```

```
class Student {
```

```
    //defining fields
```

```
    id: number;
```

```
    name:string;
```

```
    //creating method or function
```

```
    display():void {
```

```
        console.log("Student ID is: "+this.id)
```

```
        console.log("Student ID is: "+this.name)
```

```
    }
```

```
}
```

```
//Creating an object or instance
```

```
let obj = new Student();
```

```
obj.id = 101;
```

```
obj.name = "Virat Kohli";
```

```
obj.display();
```

- **Bảng Constructor**

- Constructor là một method đặc biệt được dùng để khởi tạo đối tượng của lớp. Trong TypeScript, constructor là một method đặc biệt với tên là "constructor." Trong constructor, ta có thể truy cập đến các thành phần của lớp với từ khóa **this**.

```
//Defining a Student class.
class Student {
    //defining fields
    id: number;
    name:string;

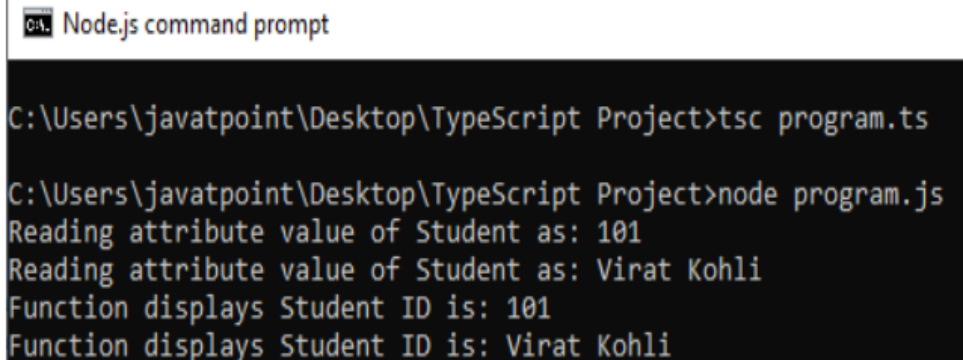
    //defining constructor
    constructor(id: number, name:string) {
        this.id = id;
        this.name = name;
    }

    //creating method or function
    display():void {
        console.log("Function displays Student ID is: " +this.id)
        console.log("Function displays Student ID is: " +this.name)
    }
}

//Creating an object or instance
let obj = new Student(101, "Virat Kohli")

//access the field
console.log("Reading attribute value of Student as: " +obj.id,)
console.log("Reading attribute value of Student as: " +obj.name)

//access the method or function
obj.display()
```



```
Node.js command prompt

C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts

C:\Users\javatpoint\Desktop\TypeScript Project>node program.js
Reading attribute value of Student as: 101
Reading attribute value of Student as: Virat Kohli
Function displays Student ID is: 101
Function displays Student ID is: Virat Kohli
```

- Interface: Là một cấu trúc đóng vai trò như một **contract** trong ứng dụng. Interface sẽ cung cấp giao diện để cho các lớp cài đặt tuân theo. Interface không thể khởi tạo và thường được dùng để cho các lớp khác cài đặt. Trình biên dịch TypeScript sẽ sử dụng interface cho quá trình **type-checking** (còn được gọi là "duck typing" hoặc "structural subtyping").
- Interface chỉ chứa khai báo của các **methods** và **fields**, và không chứa phần cài đặt (**implementation**). Các class khi cài đặt interface thì sẽ phải cài đặt tất cả các method của interface đó.
- Khi trình biên dịch TypeScript tiến hành biên dịch mã thành JavaScript, thì interface sẽ biến mất trong file JavaScript. Vì vậy interface chỉ sử dụng cho giai đoạn viết mã.
  - Để khai báo **interface** ta dùng từ khóa interface.
  - **interface\_name** là tên của interface.
  - Interface body chứa phần khai báo các biến và methods.
  - Cú pháp:

```
interface interface_name {  
    // variables' declaration  
    // methods' declaration  
}
```

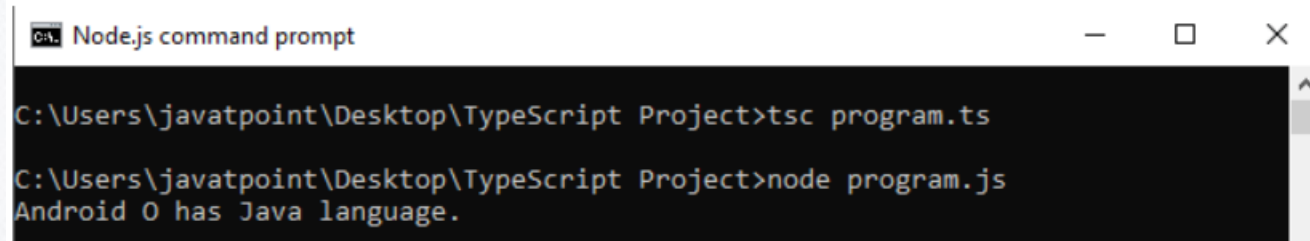


# TypeScript Interface

- Trong ví dụ sau, ta đã tạo ra một interface OS với các thuộc tính name và language thuộc kiểu string. Sau đó ta định nghĩa một function với một đối số thuộc kiểu của interface OS.

```
interface OS {  
    name: String;  
    language: String;  
}  
  
let OperatingSystem = (type: OS): void => {  
    console.log('Android ' + type.name + ' has ' + type.language + ' language.');};  
  
let Oreo = {name: 'O', language: 'Java'}  
OperatingSystem(Oreo);
```

## Output:



```
Node.js command prompt  
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
Android O has Java language.
```

- Ứng dụng của Interface

- Ta có thể sử dụng interface cho những mục đích sau:

- Truyền các Objects như các tham số
    - Trả về các Objects từ các functions.

- Interface Inheritance

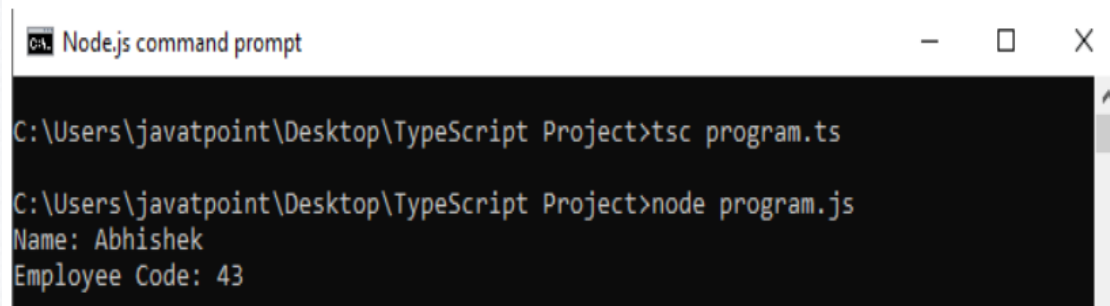
- Ta có thể kế thừa interface từ các interfaces khác bằng cách sử dụng từ khóa extends.

```
child_interface extends parent interface{  
}
```

# TypeScript Interface

```
interface Person {  
    name:string  
    age:number  
}  
  
interface Employee extends Person {  
    gender:string  
    empCode:number  
}  
  
let empObject = <Employee>{};  
empObject.name = "Abhishek"  
empObject.age = 25  
empObject.gender = "Male"  
empObject.empCode = 43  
  
console.log("Name: "+empObject.name);  
console.log("Employee Code: "+empObject.empCode);
```

## Output:



```
Node.js command prompt  
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
Name: Abhishek  
Employee Code: 43
```

- Array Type Interface

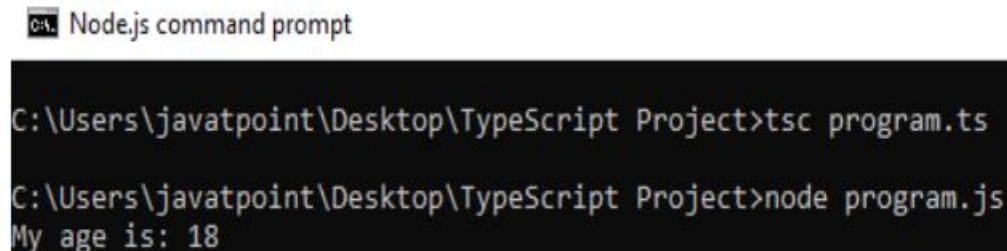
➤ Ta có thể sử dụng interfaces để mô tả về array type như ví dụ dưới đây.

```
// Array which return string
interface nameArray {
  [index:number]:string
}

// use of the interface
let myNames: nameArray;
myNames = ['Virat', 'Rohit', 'Sachin'];

// Array which return number
interface ageArray {
  [index:number]:number
}

var myAges: ageArray;
myAges =[10, 18, 25];
console.log("My age is: " +myAges[1]);
```



```
Node.js command prompt

C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts

C:\Users\javatpoint\Desktop\TypeScript Project>node program.js
My age is: 18
```



# Kế thừa (Inheritance)

- Cũng giống như các ngôn ngữ lập trình hướng đối tượng khác như Java, C#. TypeScript cho phép bạn tái sử dụng lại chức năng của một class hiện có mà không cần viết lại nó.

## Example: Inheritance

```
class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }
}

class Employee extends Person {
  empCode: number;

  constructor(empcode: number, name:string) {
    super(name);
    this.empCode = empcode;
  }

  displayName():void {
    console.log("Name = " + this.name + ", Employee Code = " + this.empCode);
  }
}

let emp = new Employee(100, "Bill");
emp.displayName(); // Name = Bill, Employee Code = 100
```

- Trong ví dụ trên, lớp Employee kế thừa lớp Person sử dụng từ khóa **Extends**. Điều này có nghĩa là lớp Employee sẽ bao gồm tất cả các thuộc tính và hàm của lớp Person

# Kế thừa (Inheritance)

- Một Class có thể implement một hoặc nhiều interface
- Trong ví dụ, lớp Employee implements 2 interface iperson và iemployee

## Example: Implement Interface

```
interface IPerson {
    name: string;
    display():void;
}

interface IEmployee {
    empCode: number;
}

class Employee implements IPerson, IEmployee {
    empCode: number;
    name: string;

    constructor(empcode: number, name:string) {
        this.empCode = empcode;
        this.name = name;
    }

    display(): void {
        console.log("Name = " + this.name + ", Employee Code = " + this.empCode);
    }
}

let per:IPerson = new Employee(100, "Bill");
per.display(); // Name = Bill, Employee Code = 100

let emp:IEmployee = new Employee(100, "Bill");
emp.display(); //Compiler Error: Property 'display' does not exist on type 'IEmployee'
```

# Trừa tượng(Abstraction)

- TypeScript cho phép định nghĩa một lớp trừu tượng sử dụng từ khóa Abstract.
- Một abstract class(lớp trừu tượng) là một lớp cơ sở, thường được sử dụng để định nghĩa các hành vi chung cho các lớp dẫn xuất(lớp kế thừa). Không giống như lớp bình thường, một lớp abstract không thể được khởi tạo trực tiếp
- Ví dụ:

```
abstract class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }

  display(): void {
    console.log(this.name);
  }
  abstract find(string): Person;
}

class Employee extends Person {
  empCode: number;
  constructor(name: string, code: number) {
    super(name); // must call super()
    this.empCode = code;
  }
  find(name: string): Person {
    // execute AJAX request to find an employee from a db
    return new Employee(name, 1);
  }
}

let emp: Person = new Employee("James", 100);
emp.display(); //James
let emp2: Person = emp.find('Steve');
```

# Trừa tượng(Abstraction)

- Trong lớp trừu tượng cũng có thể định nghĩa thuộc tính trừu tượng
- Ví dụ:

```
abstract class Person {  
  abstract name: string;  
  display(): void {  
    console.log(this.name);  
  }  
}  
  
class Employee extends Person {  
  name: string;  
  empCode: number;  
  constructor(name: string, code: number) {  
    super(); // must call super()  
  
    this.empCode = code;  
    this.name = name;  
  }  
}  
  
let emp: Person = new Employee("James", 100);  
emp.display(); //James
```

- **Chú ý:** Class mà implement một abstract class phải gọi super() trong hàm khởi tạo



# Đa hình(Polymorphism)

- Là một đối tượng thuộc các lớp khác nhau có thể hiểu cùng một thông điệp theo cách khác nhau.
- Đa hình thể hiện qua 2 tính chất cơ bản:
  - Nạp chồng (overloading),
  - Ghi đè (override)

## 1. Nạp chồng:

- Overloading method là những phương thức nằm trong cùng một class có cùng tên nhưng khác các tham số (khác kiểu dữ liệu, khác số lượng tham số)

Ví dụ:

```
//Function with string type parameter
function add(a:string, b:string): string;
//Function with number type parameter
function add(a:number, b:number): number;
//Function Definition
function add(a: any, b:any): any {
    return a + b;
}
//Result
console.log("Addition: " +add("Hello ", "JavaTpoint"));
console.log("Addition: "+add(30, 20));
```

## 2. Ghi đè:

- TypeScript cho phép một class con có thể định nghĩa, khai báo lại một phương thức (methods) từ class cha. Ví dụ:

```
class A {  
  name: string = "Class A"  
  
  print(){  
    console.log('I am class A')  
  }  
}  
  
class B extends A {  
  name: string = "Class B"  
  
  print(){  
    console.log('I am class B')  
  }  
}
```

- Là kỹ thuật được dùng để che dấu đi những chi tiết bên trong các đối tượng. Một class có thể kiểm soát phạm vi truy cập đến các thành phần của class đó đối với các lớp khác. Đặc điểm này còn được gọi là encapsulation hoặc data-hiding. Phương pháp lập trình hướng đối tượng sử dụng các access modifiers để cài đặt cơ chế encapsulation.
- TypeScript hỗ trợ các loại access modifier như sau:
  - Public: cho phép truy cập từ class ngoài
  - Private: chỉ cho phép truy cập trong Class
  - Protect: tương tự như Private. Tuy nhiên có thể truy cập từ các class kế thừa
  - Readonly: Các thuộc tính được gán với readonly thì có thể truy cập tương tự như public, tuy nhiên, giá trị của chúng thì không được phép thay đổi

- TypeScript Generics là một đặc điểm cho phép ta có thể tạo ra các component có thể tái sử dụng bằng cách cho phép các components có thể làm việc với nhiều kiểu dữ liệu khác nhau thay cho một kiểu dữ liệu duy nhất. Việc sử dụng generics cho phép chương trình có thể linh hoạt cũng như có thể mở rộng.
- Generics cung cấp cơ chế type safety mà không ảnh hưởng đến performance của ứng dụng. TypeScript uses generics, trong đó ta sẽ khai báo các type parameter với các biến hay các functions.
- Ta sẽ khai báo **type parameter** giữa cặp dấu **open (<)** và **close (>)** Generics sử dụng ký hiệu **<T>** để thể hiện **types**.
- Trong TypeScript, ta có thể tạo ra generic classes, generic functions, generic methods, và generic interfaces. TypeScript Generics rất giống với C# và Java generics.



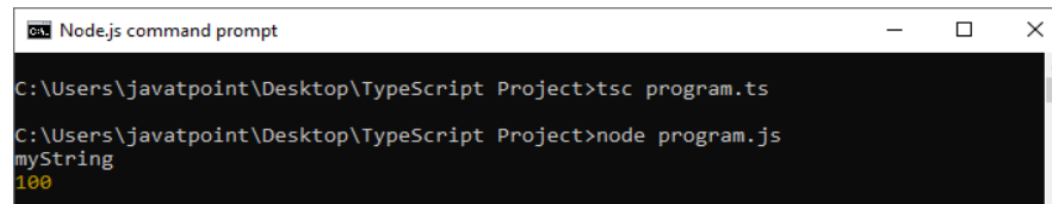
- Ví dụ:

```
function identity<T>(arg: T): T {  
    return arg;  
}  
  
let output1 = identity<string>("myString");  
let output2 = identity<number>( 100 );  
console.log(output1);  
console.log(output2);
```

- Sau khi biên dịch file trên, ta sẽ thu được file JavaScript như sau.

```
function identity(arg) {  
    return arg;  
}  
  
var output1 = identity("myString");  
var output2 = identity(100);  
console.log(output1);  
console.log(output2);
```

## Output:



```
Node.js command prompt  
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
myString  
100
```



- Lợi thế của Generics
- Generics đem lại 3 lợi ích như sau:
  - 1.**Type-safety**: Generics yêu cầu ta chỉ lưu trữ một loại đối tượng, điều này sẽ làm giảm nguy cơ gặp lỗi về sau.
  - 2.**Không cần ép kiểu**: Nhờ kiểm tra kiểu ngay lúc viết mã, nên khi lấy ra đối tượng từ collection thì sẽ không cần ép kiểu, do đó hạn chế lỗi ép kiểu sai.
  - 3.**Compile-Time Checking**: Generic cho phép phát hiện lỗi sớm lúc compile time, nhờ đó sẽ dễ sửa lỗi và hạn chế các lỗi runtime.

- Trong ví dụ trên, hàm **getItems()** sẽ nhận một mảng thuộc kiểu **any**. Hàm **getItems()** sẽ tạo ra một mảng mới thuộc kiểu **any**, nối các phần tử và trả về mảng mới.
- Khi ta sử dụng any data type, ta có thể add các phần tử thuộc kiểu bất kỳ vào trong hàm. Tuy nhiên, có thể lúc truyền dữ liệu vào ta đã truyền không chính xác, điều này sẽ dẫn đến những lỗi runtime về sau.

```
function getItems(items: any[] ) : any[] {  
    return new Array().concat(items);  
}  
  
let myNumArr = getItems([10, 20, 30]);  
let myStrArr = getItems(["Hello", "JavaTpoint"]);  
myNumArr.push(40); // Correct  
myNumArr.push("Hello TypeScript"); // Correct  
myStrArr.push("Hello SSSIT"); // Correct  
myStrArr.push(40); // Correct  
  
console.log(myNumArr); // [10, 20, 30, 40, "Hello TypeScript"]  
console.log(myStrArr); // ["Hello", "JavaTpoint", "Hello SSSIT", 40]
```

## Output:

 Node.js command prompt

```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
[ 10, 20, 30, 40, 'Hello TypeScript' ]  
[ 'Hello', 'JavaTpoint', 'Hello SSSIT', 40 ]
```

- Để giải quyết vấn đề này, TypeScript đã giới thiệu generics. Trong generics, type variable chỉ chấp nhận kiểu mà người dùng đã khai báo. Do đó, hàm có thể được viết lại như sau
- Trong ví dụ trên, type variable T được khai báo trong hàm như sau **getItems<T>**. Biến này khai báo ở tham số và giá trị trả về. Điều này sẽ đảm bảo rằng kiểu dữ liệu được chỉ định lúc gọi hàm cũng sẽ chính là kiểu dữ liệu được khai báo trong đối số và giá trị trả về từ hàm.
- Generic function **getItems()** trong ví dụ trên sẽ nhận một mảng số và mảng các chuỗi. Khi ta gọi hàm **getItems<number>([10, 20, 30])**, thì T sẽ được thay thế bởi số. Do đó, kiểu của đối số và giá trị trả về sẽ là mảng chữ số. Tương tự đối với lời gọi hàm **getItems<string>(["Hello", "JavaTpoint"])**, kiểu của đối số và giá trị trả về sẽ là mảng các chuỗi. Nếu ta cố tình thêm một chuỗi vào mảng **arrNumber** hoặc một số vào mảng **arrString**, trình biên dịch sẽ báo lỗi.

```
function getItems<T>(items : T[] ) : T[] {  
    return new Array<T>().concat(items);  
}  
  
let arrNumber = getItems<number>([10, 20, 30]);  
let arrString = getItems<string>(["Hello", "JavaTpoint"]);  
  
arrNumber.push(40); // Correct  
arrNumber.push("Hi! Javatpoint"); // Compilation Error  
arrString.push("Hello TypeScript"); // Correct  
arrString.push(50); // Compilation Error  
  
console.log(arrNumber);  
console.log(arrString);
```

## Output:

Node.js command prompt

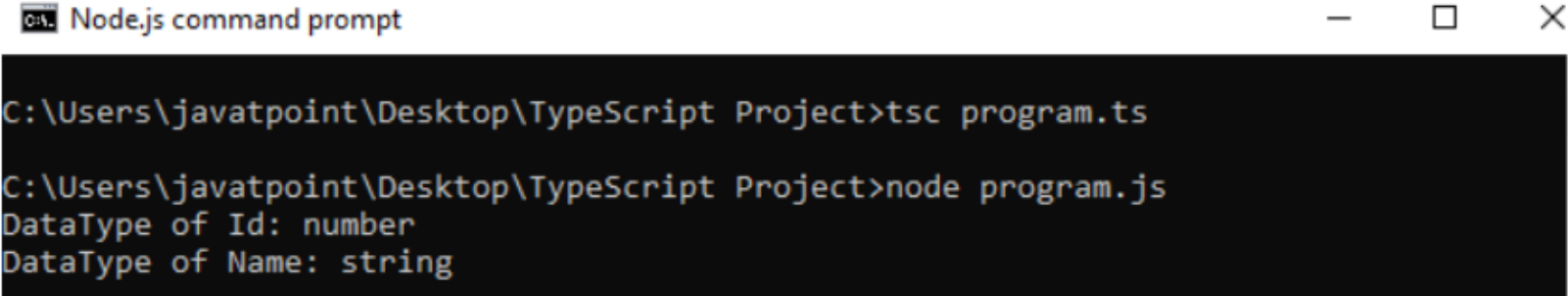
```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
[ 10, 20, 30, 40 ]  
[ 'Hello', 'JavaTpoint', 'Hello TypeScript' ]
```

- Multi-type variables

- Trong TypeScript Generics, ta có thể khai báo các multi-type variables với tên khác nhau như sau.

```
function displayDataType<T, U>(id:T, name:U): void {  
    console.log("DataType of Id: "+typeof(id) + "\nDataType of Name: " + typeof(name));  
}  
displayDataType<number, string>(101, "Abhishek");
```

## Output:



```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
DataType of Id: number  
DataType of Name: string
```

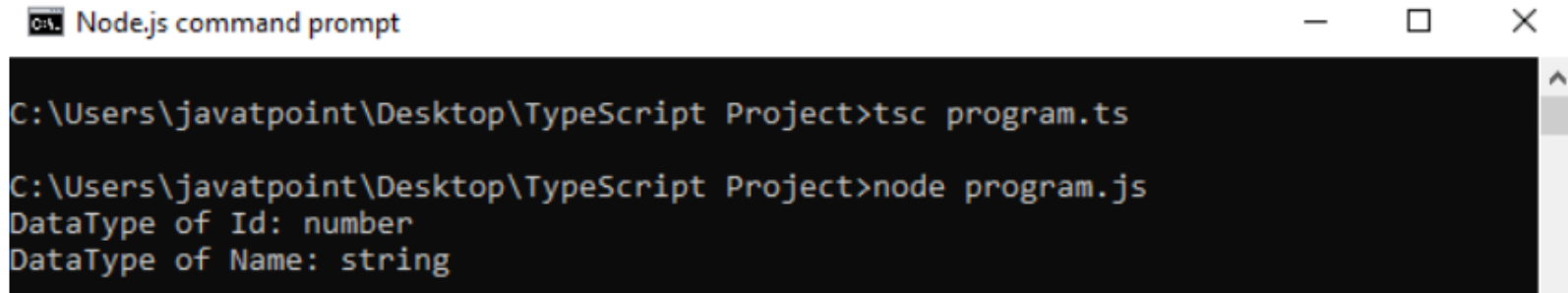


- Generic với non-generic Type

- Ta có thể sử dụng generic types với các non-generic types như sau.

```
function displayDataType<T>(id:T, name:string): void {  
    console.log("DataType of Id: "+typeof(id) + "\nDataType of Name: " + typeof(name));  
}  
displayDataType<number>(1, "Abhishek");
```

**Output:**



The screenshot shows a Node.js command prompt window with the following content:

```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
DataType of Id: number  
DataType of Name: string
```

- Generics Classes

- TypeScript cũng hỗ trợ các generic classes. Trong đó generic type parameter được chỉ định bên trong cặp dấu (<>) theo sau tên class. Một generic class có thể có các generic fields hoặc methods.

```
class StudentInfo<T,U>
{
    private Id: T;
    private Name: U;
    setValue(id: T, name: U): void {
        this.Id = id;
        this.Name = name;
    }
    display():void {
        console.log(`Id = ${this.Id}, Name = ${this.Name}`);
    }
}

let st = new StudentInfo<number, string>();
st.setValue(101, "Virat");
st.display();

let std = new StudentInfo<string, string>();
std.setValue("201", "Rohit");
std.display();
```

## Output:

 Node.js command prompt

```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts

C:\Users\javatpoint\Desktop\TypeScript Project>node program.js
Id = 101, Name = Virat
Id = 201, Name = Rohit
```

- Generics Interface

➤ Generic type cũng có thể được sử dụng với interface như sau.

```
interface People {  
    name: string  
    age: number  
}  
  
interface Celebrity extends People {  
    profession: string  
}  
  
function printName<T extends Celebrity>(theInput: T): void {  
    console.log(`Name: ${theInput.name} \nAge: ${theInput.age} \nProfession: ${theInput.profession}`);  
}  
  
let player: Celebrity = {  
    name: 'Rohit Sharma', age: 30, profession: 'Cricket Player'  
}  
  
printName(player);
```

## Output:

Node.js command prompt

```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
Name: Rohit Sharma  
Age: 30  
Profession: Cricket Player
```

- Generic Constraints

- TypeScript Generics Types cho phép làm việc với bất kỳ kiểu dữ liệu nào. Tuy nhiên, ta có thể giới hạn các kiểu dữ liệu bằng cách sử dụng constraints. Ta sử dụng từ khóa **"extends"** để biểu diễn constraint.

```
interface Lengthwise {  
  length: number;  
}  
  
function loggingIdentity<T extends Lengthwise>(arg: T): T {  
  console.log("Length: " + arg.length); // It has a .length property, so no more error found  
  return arg;  
}  
  
loggingIdentity({length: 10, value: 9});  
loggingIdentity(3); // Compilation Error, number doesn't have a .length property
```

**Output:**

```
Length: 10  
Length: undefined
```



- Generic Constraints với class

```
class Student {  
    Id: number;  
    Name: string;  
  
    constructor(id:number, name:string) {  
        this.Id = id;  
        this.Name = name;  
    }  
}  
  
function display<T extends Student>(per: T): void {  
    console.log(`${ st.Id} ${st.Name}` );  
}  
  
var st = new Student(101, "\nVirat Kohli");  
display(st);
```

## Output:

 Node.js command prompt

```
C:\Users\javatpoint\Desktop\TypeScript Project>tsc program.ts  
C:\Users\javatpoint\Desktop\TypeScript Project>node program.js  
101  
Virat Kohli
```

- Namespace là một cách thức để thực hiện cơ chế **logical grouping** của các chức năng. Namespaces sẽ đóng gói các đặc điểm và các objects có chia sẻ các mối quan hệ chung. Namespaces cho phép ta tổ chức mã nguồn theo cách thức sáng sủa hơn.
- Một namespace còn được gọi là một **internal modules**. Một namespace cũng có thể bao gồm các interfaces, classes, functions, và các biến variables để hỗ trợ một nhóm các chức năng có liên quan.
- Không giống như JavaScript, namespaces được dựng sẵn **trong** TypeScript. Trong JavaScript, việc khai báo biến sẽ có phạm vi **global scope**. Nếu nhiều file JavaScript được dùng trong cùng dự án, thì sẽ có khả năng người dùng mới sẽ bị nhầm lẫn, khai báo biến đè lên với cùng tên. Vì vậy, việc sử dụng TypeScript namespace sẽ tránh nguy cơ **naming collisions**.
- **Khai báo Namespace**
  - Để tạo namespace, ta sử dụng từ khóa **namespace** sau đó là **namespace\_name**. Tất cả các interfaces, classes, functions, và các variables có thể được định nghĩa bên trong cặp dấu **curly braces**{} bằng cách sử dụng từ khóa **export**. Từ khóa export cho phép các component có thể được truy nhập từ bên ngoài namespaces.

- Cú pháp:

```
namespace <namespace_name> {  
    export interface I1 {}  
    export class c1{}  
}
```

- Để truy cập đến các interfaces, classes, functions, và các variables trong namespace khác, ta sử dụng cú pháp như sau:

```
namespaceName.className;  
namespaceName.functionName;
```

- Nếu namespace được chứa trong một file TypeScript khác, thì cần phải được tham chiếu bằng cách sử dụng ký tự sau **triple-slash (///)**.

```
/// < reference path = "Namespace_FileName.ts" />
```

- Module là cách thức để tạo ra một nhóm các variables, functions, classes, và interfaces có liên quan đến nhau. Module thực thi trong **local scope**, không phải là **global scope**. Hoặc ta có thể hiểu rằng các variables, functions, classes, và interfaces được khai báo bên trong một module không thể được truy cập trực tiếp từ bên ngoài module. Ta có thể tạo ra một module với từ khóa **export** và sau đó có thể sử dụng chúng trong các modules khác bằng cách sử dụng từ khóa **import**.
- Modules import module khác bằng cách sử dụng một **module loader**. Tại giai đoạn runtime, module loader sẽ phụ trách việc định vị và thực thi tất cả các dependencies của một module trước khi thực thi module đó. Modules loaders phổ biến nhất được sử dụng trong JavaScript là **CommonJS** module loader dùng cho **Node.js** và **require.js** dùng cho các Web applications.
- Module gồm 2 loại sau:
  1. Internal Module
  2. External Module



- Internal Module

- Internal modules được dùng trong các phiên bản trước đây của Typescript. Được dùng cho quá trình **logical grouping** các classes, interfaces, functions, variables thành một đơn vị duy nhất và có thể được exported vào một module khác. Các modules được sử dụng như **namespace** có trong các phiên bản mới nhất của TypeScript. Hiện nay các internal modules đã được khai báo là **obsolete**. Tuy nhiên chúng vẫn được hỗ trợ dưới dạng các namespace.

- Cú pháp Internal Module trong các phiên bản cũ:

```
module Sum {  
    export function add(a, b) {  
        console.log("Sum: " + (a+b));  
    }  
}
```

- Cú pháp Internal Module từ phiên bản ECMAScript 2015:

```
namespace Sum {  
    export function add(a, b) {  
        console.log("Sum: " + (a+b));  
    }  
}
```

- External Module

- External modules cũng còn được gọi là **module**. Khi một ứng dụng bao gồm rất nhiều file, ta sẽ cần một cách thức để quản lý hiệu quả các file này. External Module được dùng để thiết lập **load dependencies** giữa các file external js files.

- Khai báo Module

- Ta có thể khai báo một module với từ khóa **export** theo cú pháp sau.

```
//FileName : EmployeeInterface.ts  
export interface Employee {  
    //code declarations  
}
```

- Ta có thể khai báo module trong các files khác bằng từ khóa **import**. Tên **file/module** được chỉ định mà không có phần mở rộng (**extension**).

```
import { class/interface name } from 'module_name';
```

- Định nghĩa module

```
export class Addition{  
    constructor(private x?: number, private y?: number){  
    }  
    Sum(){  
        console.log("SUM: " +(this.x + this.y));  
    }  
}
```

- Sử dụng module trong file khác với từ khóa import : **app.ts**

```
import {Addition} from './addition';  
  
let addObject = new Addition(10, 20);  
  
addObject.Sum();
```

- Biên dịch và thực thi các Modules
  - Mở **terminal** và chuyển đến thư mục nơi chứa **project**. Sau đó gõ lệnh sau trong terminal window.

```
$ tsc --module commonjs app.ts  
$ node ./app.js
```

**Output:**

```
Sum: 30
```



- Importing nhiều modules trong một file
  - Ta có thể import nhiều modules trong một file như sau.

```
import { export_name1, export_name2 } from 'module_name';
```

- Ví dụ:

```
export class Addition{
    constructor(private x?: number, private y?: number){
    }
    Sum(){
        console.log("SUM: " +(this.x + this.y));
    }
}
export class Substraction{
    constructor(private a?: number, private b?: number){
    }
    Substract(){
        console.log("SUBSTRACTION: " +(this.a - this.b));
    }
}
```

- Sau đó ta có thể sử dụng module trong file khác với từ khóa import : **app.ts**

```
import {Addition, Substraction} from './Modules';

let addObject = new Addition(10, 20);
let subObject = new Substraction(20, 10);

addObject.Sum();
subObject.Substract();
```

- Biên dịch và thực thi Multiple Modules
  - Mở **terminal** và chuyển đến thư mục chứa dự án, sau đó gõ lệnh sau trong terminal window.

```
$ tsc --module commonjs app.ts
$ node ./app.js
```

**Output:**

```
SUM: 30
SUBTRACTION: 10
```

- Re-exports

- Trong TypeScript đôi khi các modules có thể mở rộng các modules khác, và có thể expose một vài tính năng của các module khác thông qua cơ chế re-export. Ta có thể đổi tên module được re-export.
- Trong ví dụ trên, tên của lớp **Addition** đã được đổi thành **plus** với cú pháp {Addition as plus}. Lệnh re-exporting sẽ hữu ích khi ta gán một tên có ý nghĩa cho module.

## Module Creation: **Modules.ts**

```
export class Addition{  
    constructor(private x?: number, private y?: number){  
    }  
    Sum(){  
        console.log("SUM: " +(this.x + this.y));  
    }  
}
```

## Create re-exports module: **re-exports.ts**

```
// Re-exporting types Addition as plus from Modules file  
export { Addition as plus } from "./Modules";
```

- Ta có thể truy nhập đến trong file khác bằng cách sử dụng từ khóa import: **app.ts**

```
//importing the exporting types from re-exports file
import {plus as Addition} from './re-exports'; //importing plus as Addition

let addObject = new Addition(10, 20);

addObject.Sum();
```

- Biên dịch và thực thi Modules

➤ Mở **terminal** rồi chuyển đến thư mục chứa dự án, sau đó gõ lệnh sau.

```
$ tsc --module commonjs app.ts
$ node ./app.js
```

**Output:**

```
Sum: 30
```



- TypeScript cung cấp cách thức an toàn và dễ dàng để sử dụng các thư viện JavaScript đang tồn tại như jQuery, AngularJS, Node.js v.v... thông qua khai báo Ambient.
- Khai báo Ambient sẽ thông báo cho trình biên dịch TypeScript về mã nguồn thực tế (như các variable/functions) đang tồn tại ở đâu đó. Nếu mã TypeScript cần sử dụng một third-party library được viết bằng các thư viện JavaScript thuần như jQuery/AngularJS/Node.js, thì ta có thể viết các khai báo ambient.
- Các file khai báo Ambient cần được lưu với phần mở rộng là (**d.ts**). Một file với phần mở rộng .d.ts phải có từ khóa **declare** trước định nghĩa thẻ gốc. Nếu mã nguồn không tồn tại lúc runtime thì sẽ kết thúc mà không có cảnh báo.
- Các file khai báo ambient cũng giống như các file **docs**. Nếu mã nguồn thay đổi, file docs cũng cần được cập nhật. Nếu file khai báo ambient không được cập nhật thì sẽ có lỗi biên dịch.

- Cú pháp để khai báo ambient như sau:

```
declare module module_name{  
}
```

- Cú pháp để truy cập file ambient

```
/// <reference path = "AmbientFileName.d.ts" />
```

- Bài 1

Viết một chương trình cho phép định nghĩa lớp. Tạo ra một lớp Calculator thể hiện máy tính điện tử. Định nghĩa 4 method add(), minus(), devide(), multiple() cho phép thể hiện 4 phép tính cơ bản.

Tiếp theo hãy khởi tạo một đối tượng của lớp, rồi gọi 4 method của lớp để xem kết quả. Chú ý: Hai số được khai báo và gán giá trị.

- Lớp và đối tượng
- Lập trình hướng đối tượng với TypeScript
- Interface
- Generics
- Namespace & Module
- Ambient